

PROMPT MAESTRO — App educativa de IA en un único fichero HTML

Cómo usarlo: rellena solo el BLOQUE 0 con los datos de tu nueva app y pega el prompt completo.
Todo lo demás es fijo y no necesitas tocarlo.

BLOQUE 0 · CONFIGURACIÓN DE ESTA APP (rellenar)

- **Nombre de la app:** [ej. NutriQuest]
 - **Objetivo en una frase:** [ej. generar exámenes personalizados a partir de un temario]
 - **Usuario final:** [ej. estudiantes de 3º de Medicina / profesores de FP / residentes de urgencias]
 - **Rol del modelo (system prompt):** actúa como [rol experto], con tono [didáctico / clínico / riguroso...], siguiendo siempre estas reglas: [reglas, idioma, nivel, qué no debe hacer]
 - **Campos del formulario:** [lista: nombre del campo - tipo (input/textarea/select) - obligatorio si/no - valor por defecto si aplica]
 - **Formato exacto de salida:** [texto libre / markdown / JSON con este esquema: {...} / tabla]
 - **Proveedor por defecto:** [OpenAI / Anthropic / Google] — **Modelo por defecto:** [ej. gpt-4o-mini]
 - **Extras opcionales:** [exportar a Word con docx.js / exportar a PDF con jsPDF / historial en IndexedDB / gamificación / voz con Web Speech API / ninguno]
-

INSTRUCCIONES (fijas — no modificar)

Actúa como ingeniero de producto senior + prompt engineer. Vamos a construir la app definida en el BLOQUE 0 como **un único fichero HTML autocontenido** (HTML + CSS + JS vanilla, sin frameworks, sin build, sin backend), listo para desplegar en GitHub Pages. La API se llama **directamente desde el navegador** con la api key del propio usuario.

Trabaja en este orden:

FASE 1 · Especificación (antes de escribir código)

Devuélveme, en máximo media página, tu propuesta de:

1. Objetivo refinado en una frase.
2. System prompt completo y literal que usarás (basado en el rol del BLOQUE 0).
3. Lista definitiva de campos del formulario con tipo, obligatoriedad y placeholder.
4. Formato de salida exacto. Si es JSON, define el esquema completo con un ejemplo.
5. Casos límite que vas a cubrir: campos vacíos, fallo de API, respuesta con formato inesperado, respuesta truncada.

Si algo del BLOQUE 0 es ambiguo, propón tú la mejor opción y márcala como suposición. **No me hagas preguntas: decide y continúa.**

FASE 2 · Interfaz y arquitectura

Genera el fichero HTML con esta estructura obligatoria:

1. **Gestión de api keys multi-proveedor:**
 - Selector de proveedor: OpenAI / Anthropic / Google Gemini.
 - Selector de modelo, dependiente del proveedor, con los modelos recientes de cada uno y el modelo por defecto del BLOQUE 0 preseleccionado. Incluye siempre una opción "Otro..." con input libre para escribir cualquier identificador de modelo.
 - Campo tipo `password` para la api key del proveedor activo, con botones **Guardar** y **Borrar**.
 - Persistencia en `localStorage` con estas claves exactas: `ia_openai_key`, `ia_anthropic_key`, `ia_gemini_key`. La key **nunca** aparece en el código fuente ni se envía a ningún servidor que no sea el endpoint oficial del proveedor.
2. **Formulario** con los campos definidos en la Fase 1.
3. **Botón "Generar"** deshabilitado hasta que exista api key guardada para el proveedor activo y los campos obligatorios estén completos.
4. **Área de resultado** con spinner de carga y renderizado del formato de salida (si es markdown, renderízalo; si es JSON, píntalo como interfaz, no como texto crudo).
5. **Botones de exportación:** copiar al portapapeles + los extras del BLOQUE 0 (docx.js y/o jsPDF desde CDN si se han pedido).
6. Diseño limpio, responsive, en español, con el nombre de la app como cabecera.

FASE 3 · Llamada real a la API

Implementa `async function generar()` con una **capa de abstracción única** `llamarIA(systemPrompt, userPrompt, {proveedor, modelo, apiKey})` que enrute según el proveedor:

- **OpenAI** → `POST https://api.openai.com/v1/chat/completions` · header `Authorization: Bearer <key>` · body `{model, messages:[{role:"system",...},{role:"user",...}]}` · respuesta en `choices[0].message.content`.
- **Anthropic** → `POST https://api.anthropic.com/v1/messages` · headers `x-api-key: <key>`, `anthropic-version: 2023-06-01`, `anthropic-dangerous-direct-browser-access: true`, `content-type: application/json` · body `{model, max_tokens, system, messages:[{role:"user",...}]}` · respuesta en `content[0].text`.
- **Google Gemini** → `POST https://generativelanguage.googleapis.com/v1beta/models/<modelo>:generateContent` · header `x-goog-api-key: <key>` · body `{system_instruction:{parts:[{text:...}]}, contents:[{parts:[{text:...}]}]}` · respuesta en `candidates[0].content.parts[0].text`.

Además:

1. Construye el mensaje de usuario a partir del formulario con etiquetas claras por campo.
2. Gestiona el estado de carga: botón deshabilitado + spinner mientras dura la petición.
3. Si el formato de salida es estructurado (JSON) y la respuesta no parsea: reintenta **una sola vez** enviando la respuesta defectuosa al modelo con la instrucción de devolver solo el formato correcto, sin texto adicional ni fences de markdown.
4. Muestra al pie un contador aproximado de tokens de entrada/salida y coste estimado según el modelo usado (tabla de precios como constante editable en el código).

FASE 4 · Robustez y seguridad

1. Valida el formato de la key antes de llamar (**sk-** para OpenAI, **sk-ant-** para Anthropic, **AIZA** para Gemini) para no gastar peticiones con claves claramente erróneas.
2. Captura y muestra con mensajes claros en español: 401/403 (key inválida), 429 (límite de uso), 404 (modelo inexistente), error de red, y timeout de 60 segundos con **AbortController**.
3. El botón **Borrar** elimina la key del **localStorage** y de memoria al instante.
4. Aviso visible permanente: *"La IA puede cometer errores. Revisa el resultado antes de usarlo; esta herramienta no sustituye el criterio profesional."*
5. Comentario inicial en el HTML con instrucciones para cualquier profesor: cómo obtener una api key de cada proveedor (URLs de las tres consolas), cómo publicar el fichero gratis en GitHub Pages, y qué constantes editar para adaptar el system prompt, los modelos y los precios a su asignatura.

FASE 5 · Entrega

Dame el **fichero HTML final completo en un solo bloque**, sin fragmentos omitidos ni `"/` resto igual", listo para guardar como **index.html** y subir a GitHub Pages. Antes del código, un resumen de 3 líneas de lo que hace la app. Después del código, una lista breve de ideas de mejora futura.

Reglas transversales (siempre)

- Un solo fichero. Sin backend. Sin dependencias salvo CDNs (docx.js, jsPDF, marked... solo si se necesitan).
- La api key nunca viaja a ningún servidor propio ni de terceros: solo al endpoint oficial del proveedor.
- Todo el texto de la interfaz en español.
- Código comentado en las partes que un profesor querrá adaptar (system prompt, modelos, precios, campos).